



Improving Quality Control and Efficiency in Corrugated Board Production

Our Team



Ayan Raheel



Colleen Jung



Nia Gangar



Thomas

Accomplishments & Next Steps



Built a sample app to tackle the raw material and processing problems



Built a power automate workflow for the alerting mechanism



Integrating the model into the power automate workflow



Getting a real-time video feed and using it in our power app



Understanding how to score our model based on outputs generated



Improve the model by feeding more videos

Executive Summary

Background

Corrugated Supplies, LLC, a company specializing in the production of corrugated sheets used as raw materials for typical home-delivery boxes, faces significant challenges in detecting and resolving malfunctions and quality issues during the manufacturing process.

Problem Statement

To address this, our research aims to integrate computer vision techniques and large language models into the manufacturing procedures, thereby enhancing the efficiency and effectiveness of detecting and fixing these issues

Objective

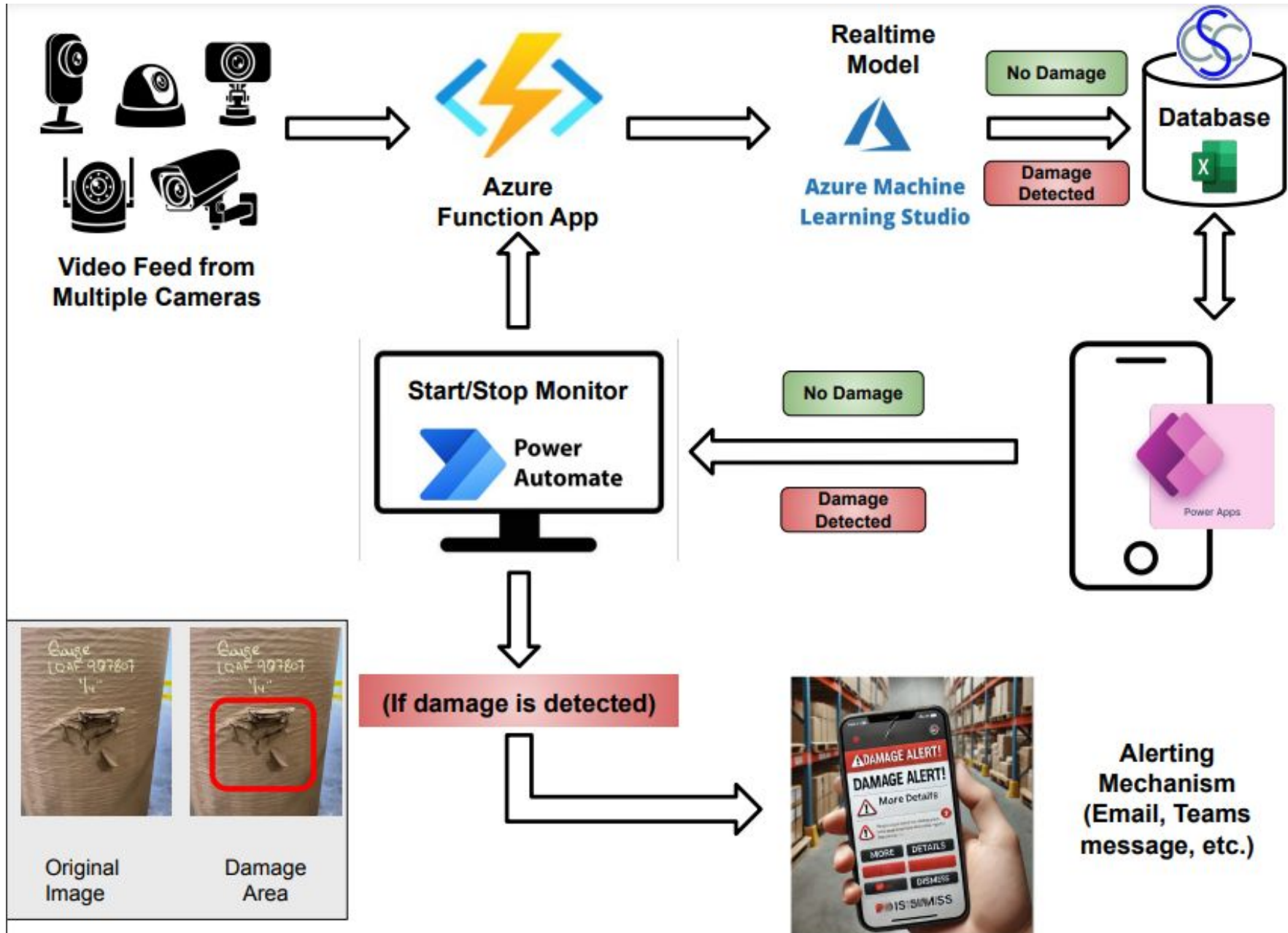
To develop a sensor that can measure the number of layers of raw materials that are damaged
To develop a real-time computer vision model that can detect the malfunction in the manufacturing procedures that may result in damaged products.

Mythology

Variance Measurement: It helps in understanding the overall variations and dynamics in the video content. High variance indicates larger changes in the pixel values over time. In our case, it implies potential malfunctions.

Neural Network Classification model: Label the pictures of damaged final products and regular products, and train a neural network classification model on Yolov8. The model will be able to distinguish the damaged products from the regular product.

How the alerting system will work



Challenges and Potential Solutions

Process	Current Problem	Solution
Raw Material Handling	Raw Material Issues Manual Documentation: time-consuming process for claiming credits for damaged material	Automated Claims Streamline the credit claim process with automated measurements and reporting
	Process Issues Imperfection, Moisture, Adhesion Problems: Waves, bubbles, dents, delamination in the material, bond failures	Improved Quality Control Correct material imperfections in real-time

Raw Material and Quality Issues



TEARS



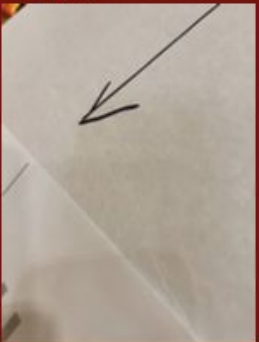
IMPRESSIONS



STREAKS



LINES



LINES



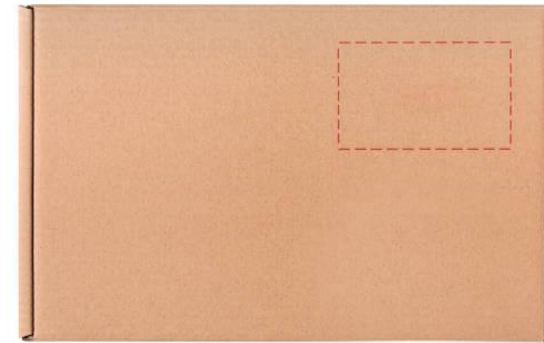
BOND ISSUE



WAVES

Dimension Measurement - TensorFlow + OpenCV

- Utilizing **OpenCV** or **TensorFlow** model to come up with our Computer Vision solution.
- First Step - **Dimension Calculation/Measurement** for the corrugated boxes/paper.
- Difference between 2D and 3D box for measurement:
 - **Length x Height**
 - **Length x Height x Breadth**
- We fed sample images to our code



How TensorFlow and OpenCV Work?

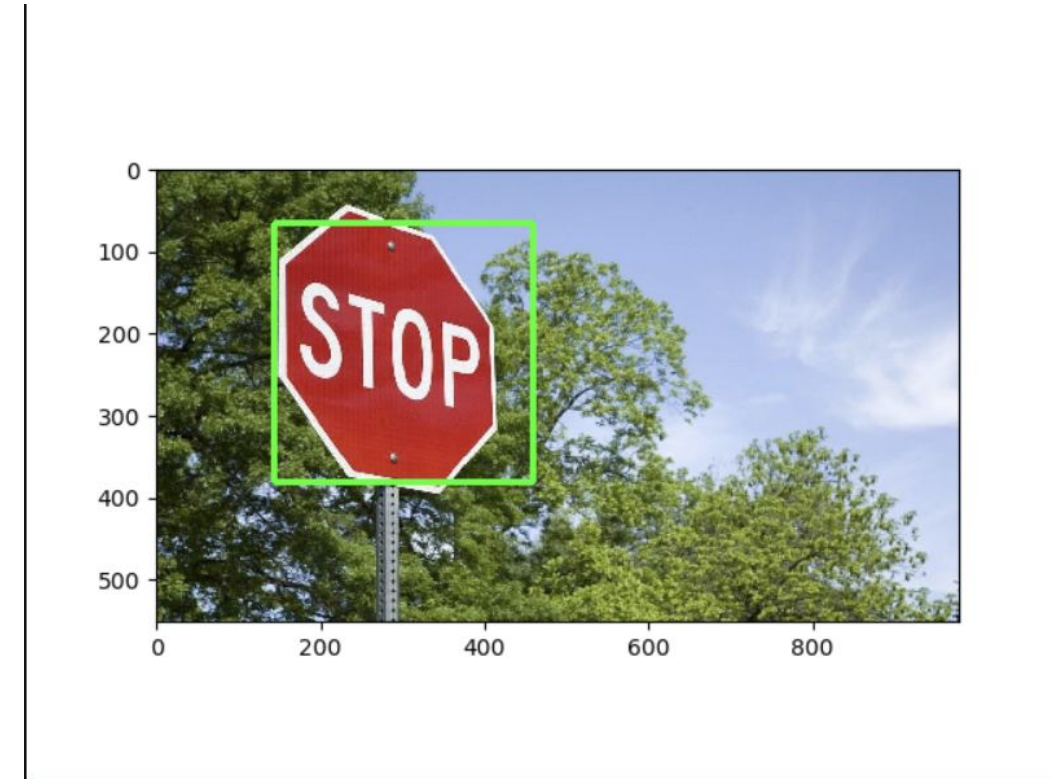
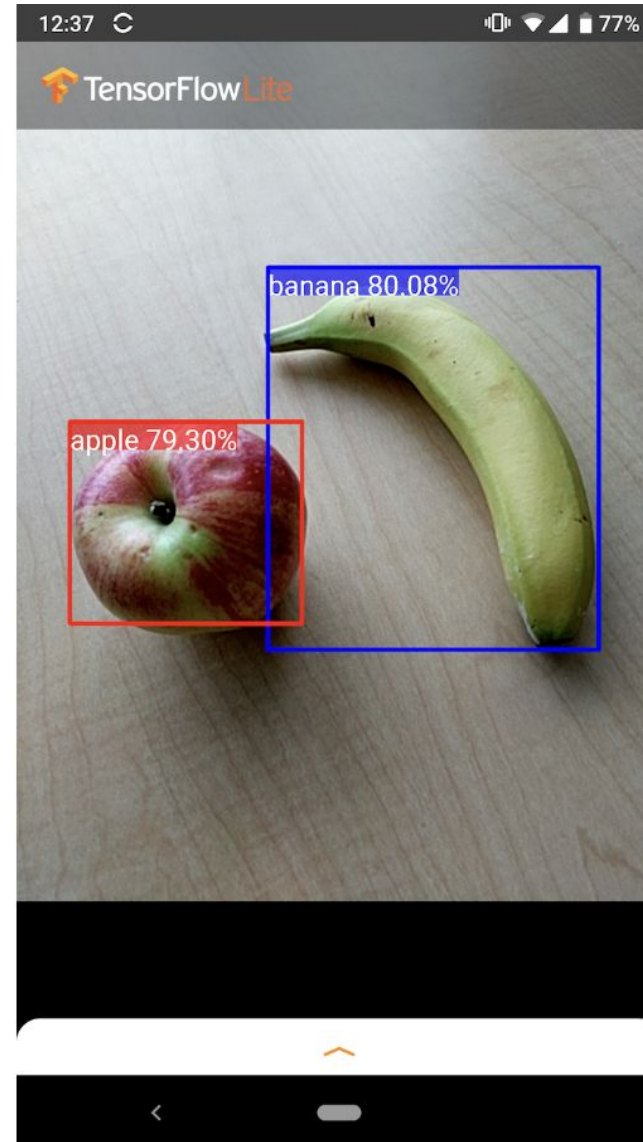
TensorFlow and OpenCV

have packages for Object Detection and Image Segmentation that pick up on anomalies

A **Bounding Box** on the Object is created that gives back a Dimensional Measurements and gives a **Confidence Interval** based on a given **Threshold**

Another Possible Solution (Automated) can be **Roboflow**

- Create a Project
- Upload + Label Data
- Deploy Model w/ Reference Object

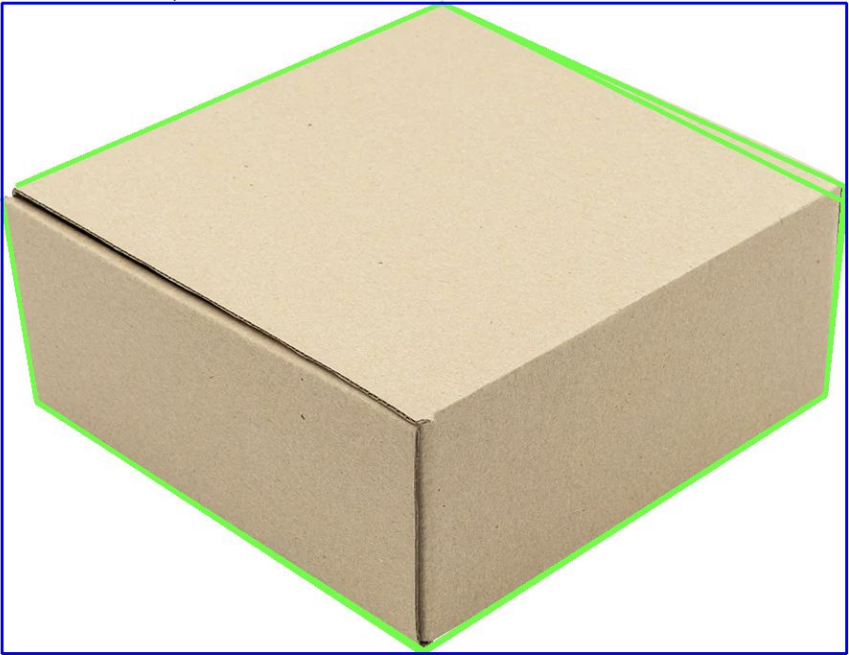


Dimension Measurement for the Corrugated Box - Output

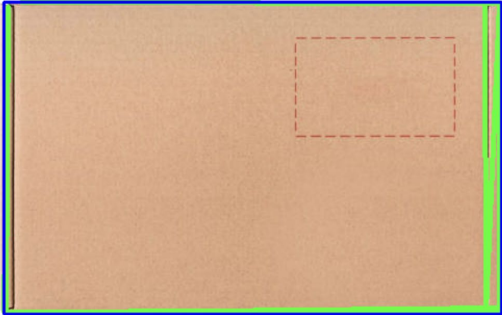


Output included a width and height for the box in pixels per metric

Width: 681px
Height: 678px



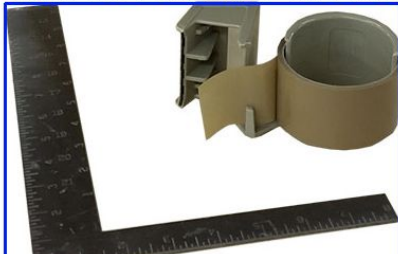
Width: 478px
Height: 300px



Dimension Measurement for the Corrugated Box with a Reference Object- Output



Length: 60.16 cm, Height: 41.45 cm, Breadth: 41.45 cm



Reference object: 20 x 20cm Ruler

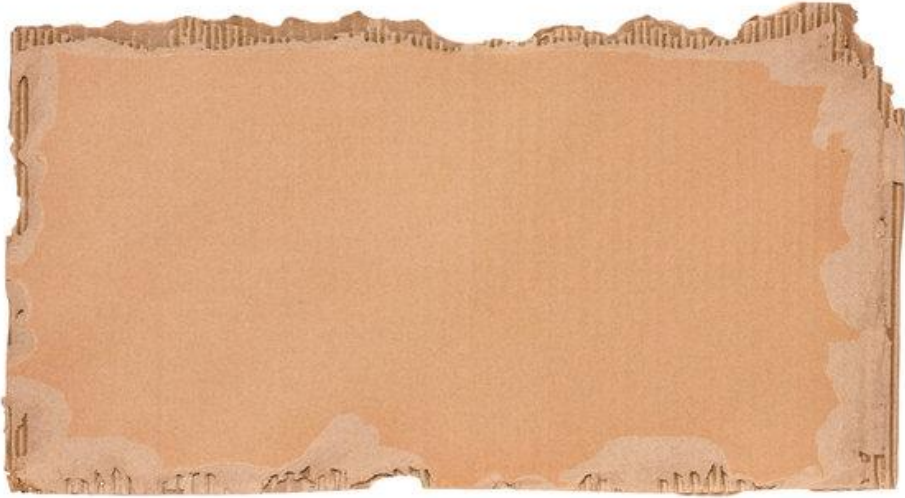
Output included a
width x height x Breadth for the box in
pixels per metric:

$\text{box_length_cm} = w / \text{pixels_per_cm}$

$\text{box_height_cm} = h / \text{pixels_per_cm}$

$\text{box_breath_cm} = b / \text{pixels_per_cm}$

Next Steps - Anomaly Detection (Early Work)

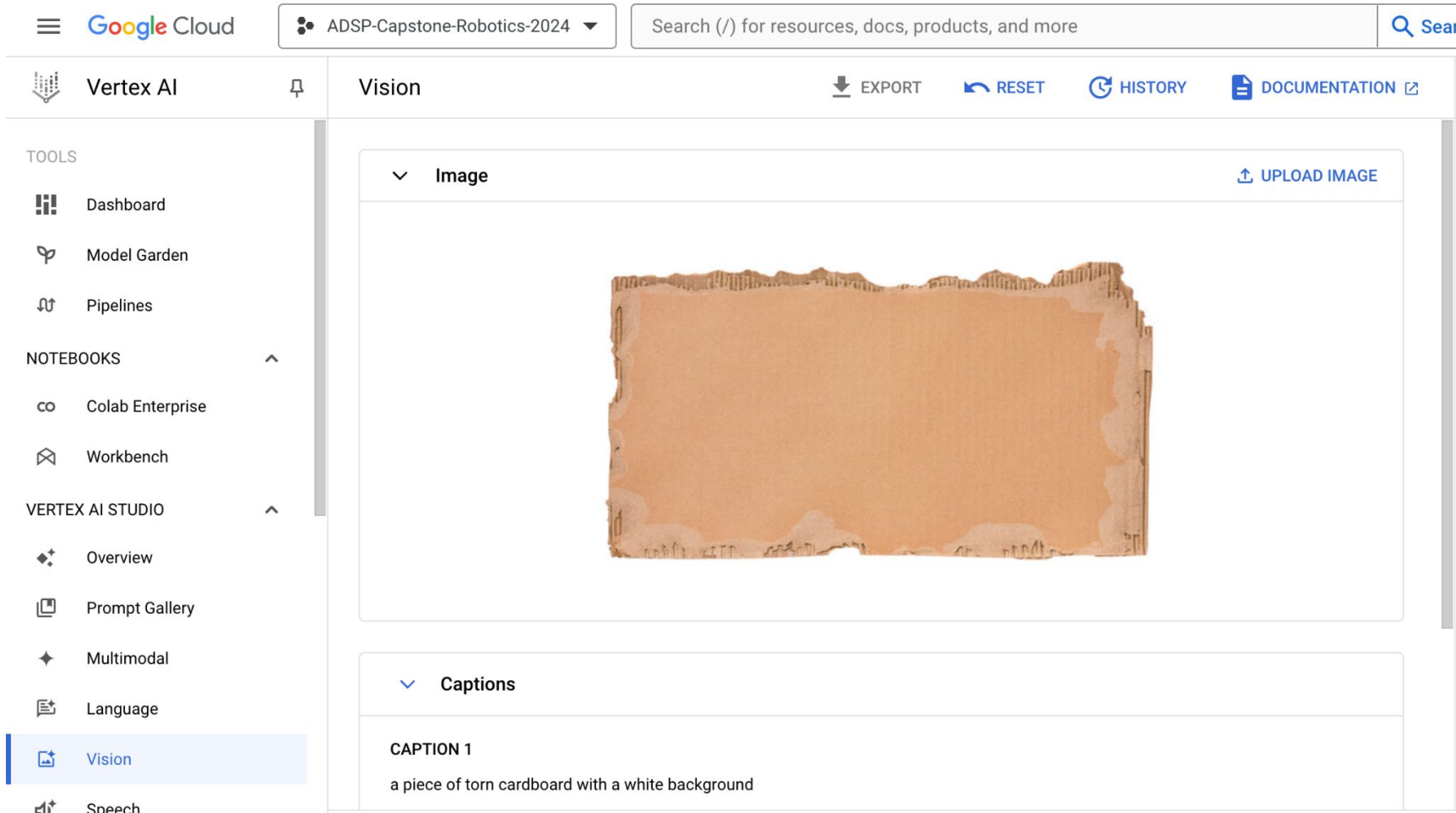


- Parsed a piece of corrugated cardboard (bad example)
- Tears on the edges + irregular dimensions



- Output showed accurate detection of the edges as anomalies (green lines)

Google Cloud Platform Vision Option



- We also utilized GCP Vertex AI Vision Option.
- Not very applicable for our solution, but can be useful in understanding underlying problems that our corrugated paper/box faces
- e.g. Any anomaly we might have missed.
- Output is only a caption that includes a description of the image that is fed to GCP Vision.

Image Raw Material 01 - Label Extraction for the Paper Rolls

PyTesseract

- Optical Character Recognition tool used for Python.
- Purpose - recognize and read in text which is embedded in pictures and images.
- We utilized the Pillow Imaging Library.
- Output was insufficient and needed substantial work.



EasyOCR

- Another Python Optical Character Recognition (OCR) tool.
- Primarily used for data entry automation and image analysis and processing.
- We preprocessed images through edge detection and contouring and fed the images to the algorithm.
- Output was more optimized and gave an Accuracy rating with a Confidence Interval of 80-90% on average.



Image Raw Material 01- Sample Output and Block

Sample Output ->

MBJE182124
(Confidence: 0.6688535764296439)

Other Noise Aspects ->

Text 7: 109
(Confidence: 0.1702106657922902)

Text 8: JCO
(Confidence: 0.08214516627997637)

Text 9: #nEHNN
(Confidence: 0.028869761126659222)



Image Raw Material 01- Sample Output 2 (Better Resolution and Image)

Sample Output ->

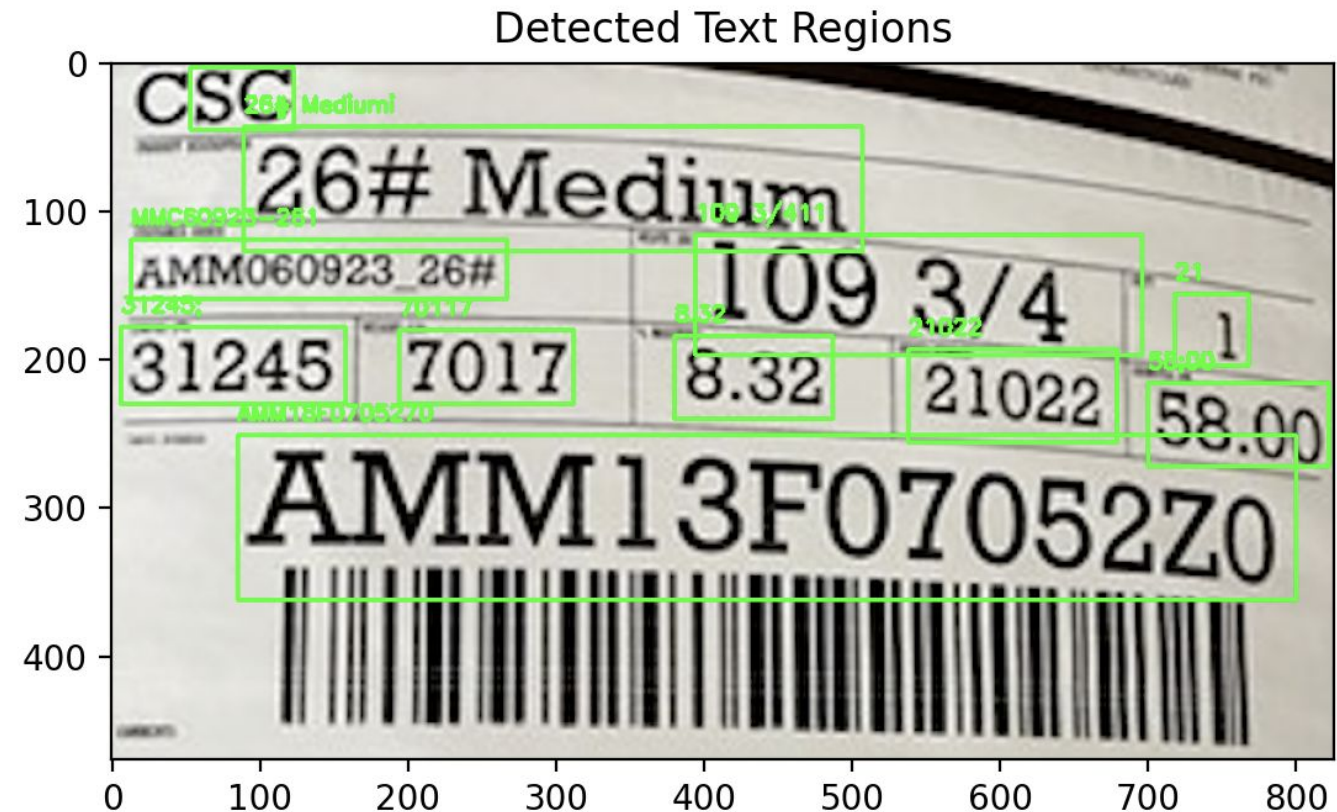
Text 2: 26# Mediumi
(Confidence: 0.5587024125117739)

Text 6: 31245;
(Confidence: 0.9512503214975172)

Text 7: 70117
(Confidence: 0.9604244882931892)

Text 9: 21022
(Confidence: 0.9748314152132349)

Text 11: AMM18F0705270
(Confidence: 0.5182913152388959)



Blockers + Suggestions for Paper Rolls

- Need to reduce the noise in the images.
- Require a standard/consistent positioned camera for images - this can allow us to custom train our model.
- Custom training can happen in the form of ROI (Range of Interest);
 - A possible solution to our problem is to hardcode the areas or regions within the tag that need to be extracted in text form for the label extraction process.
 - Through this we can reduce the noise and inaccurate extractions and convert the text to a consistent excel sheet where the information will be stored.
- Another possible solution is to keep the noise element, however, to provide us with good quality images that can be easily discerned by the computer vision system/solution.
- An example of the quality of the images (tags) is given on the left where when we Zoom in, the quality is obscured.



Sampling Box Labels to Figure Out Efficiency Levels

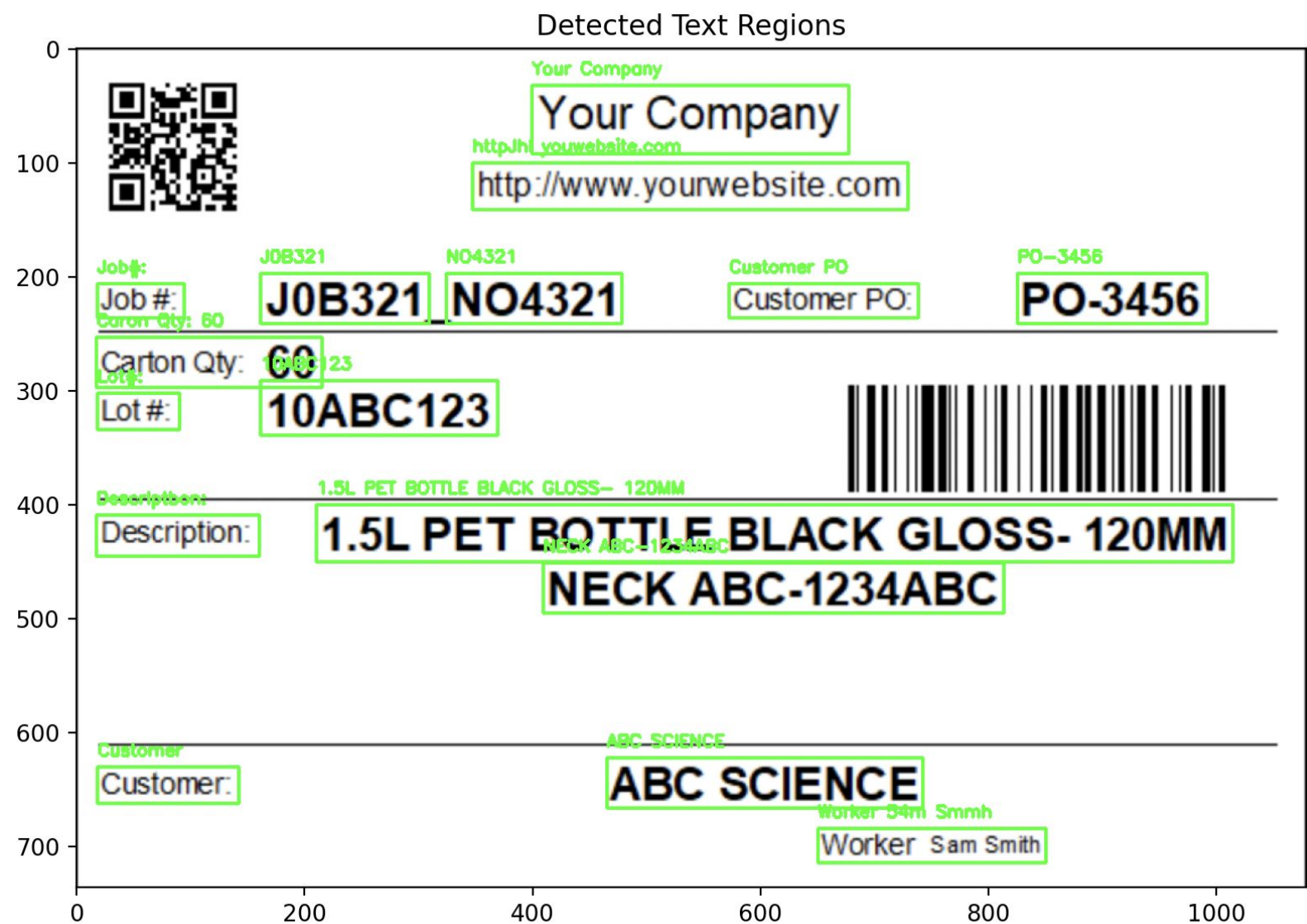


Image Raw Material 02- Damage Classification into 4 Categories

0: Unknown

1: Busted/Gouge

2: Crushed Core

3: Wet/Foreign Substances

?



Image Raw Material 02- Sample Output(prediction.csv)

Image	Category	Prediction	Confidence
MB4E25071Z - Damage.jpg	Busted:Gouged	Wet/Foreign Substances	0.40303937
V64E23091A - Damage.jpg	Busted:Gouged	Wet/Foreign Substances	0.42484212
AMM13F07052Z0 - Damage.jpg	Busted:Gouged	Wet/Foreign Substances	0.39194417
MB3E29192B - Damage.jpg	Busted:Gouged	Busted/Gouged	0.47432429
MB4B07241B - Damage.jpg	Busted:Gouged	Wet/Foreign Substances	0.44271639
MB3E18212A - Damage.jpg	Busted:Gouged	Wet/Foreign Substances	0.41944668
AMM14E25033B0 - Damage.jpg	Wet:Foreign Substance	Wet/Foreign Substances	0.44504479
AMM14E25043B0 - Damage.jpg	Wet:Foreign Substance	Wet/Foreign Substances	0.44398952
AMM14E23163B0 - Damage.jpg	Wet:Foreign Substance	Wet/Foreign Substances	0.47595695
MB4E10023B - Damage.jpg	Wet:Foreign Substance	Wet/Foreign Substances	0.57901102
P24B17242Z - Damage.jpg	Crushed Core	Wet/Foreign Substances	0.41944566
P22M21201B - Damage.jpg	Crushed Core	Wet/Foreign Substances	0.41270289

Bad Prediction and Confidence rate of approx. 0.4777

Blockers + Suggestions

Paper Roll Damage Classification

- **Need more damage image** to improve prediction accuracy
(Received 20 images -Transmit Damage)

Paper Roll Damage Measurement

- Need to **improve Label recognition** and map with Material_Defect_Data. xlsx Excel file
- Ask if the final output should be similar to the Excel file given

Moving on to Process Issues

- Need to address Paper Defect file and image



Using GPT4o (Large Language Model) for Label Extraction

- Set up Azure OpenAI Service for GPT-4O (multimodal model that leverages vision capabilities along with language understanding)
- Create an Azure OpenAI Resource and obtain the API credentials to be called in the code
- We will use the OpenAI and Pillow libraries
- Preprocessing the image to ensure it is in the right format and up to the quality standards (possibly using contouring and edge detection to better quality from our end)
- Write and execute the code for label extraction
- Within the code embed a prompt that is to be parsed
 - Sample prompt - “Extract the dimensions and box number from the image”



Best Practices for Label Extraction

- Inspect image quality and tailor the resolution to the specific requirements - the code will read whatever the human eye can view.
- The Prompt needs to be concise and accurate for the information to be read - must know what all we need to extract.
- Manage the extraction within an excel file by saving it there.
- Using a baseline model like PyTesseract or EasyOCR to compare and contrast the results. This model can also be used as a base layer that is to be built underneath the Multimodal model layer.
- Ensure an Azure instance and API credentials are valid and can be utilized.
- Manage User access.



Significant Blockers This Week + Sample Code

```
# Set up OpenAI API credentials
openai.api_key = "your-api-key"
openai.api_base = "https://your-endpoint-url"
openai.api_version = "2023-05-15"

# Function to send an image to the GPT-4O model
def extract_labels_from_image(image_path):
    # Load and prepare the image
    with open(image_path, "rb") as image_file:
        image_data = image_file.read()

    # Create a prompt for the model
    prompt = "Extract the text and labels from this image."

    # Call the GPT-4O model
    response = openai.Image.create(
        model="gpt-4o",
        prompt=prompt,
        images=[{
            "data": image_data,
            "name": "tag_image"
        }]
    )

    # Process and return the response
    labels_and_text = response['data']['text']
    return labels_and_text

# Example usage
image_path = "/path/to/your/image.jpg"
extracted_text = extract_labels_from_image(image_path)
print("Extracted Text and Labels:", extracted_text)
```

Power Apps

Using Power Apps to Manage Product Damage Reports

1. **Data Source Configuration:** Choose & Connect Data Source: (SharePoint, OneDrive, Azure Blob Storage)
2. **App Interface Design** - Forms & Media Controls: Capture details, upload images/videos
3. **Display & Analyze Data**
 - Gallery Control: Show uploaded media
 - Detail Screen: Detailed view of selected items
4. **Intelligent Features**
 - AI Integration: Detect & categorize damage
 - Automation: Use Power Automate for workflows

Alerting System

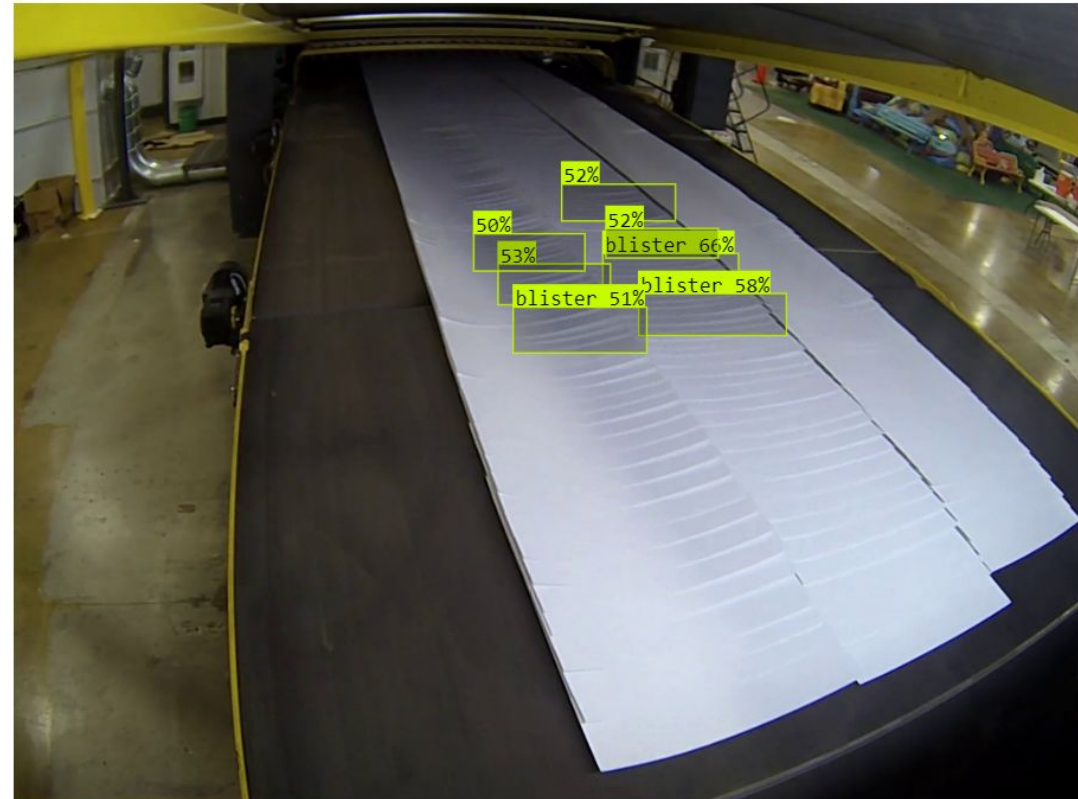
- Embed the trained model on the camera and the model will return a json file
- The json file on the right includes a confidence level. We could set a threshold based on the confidence level for sending alarm
- We recommend using the empirical probability of being damaged as the threshold
- When the threshold is passed, an alarm will be triggered and emails will be sent.
- To avoid false positives, we will implement additional checks or a buffering system that requires multiple consecutive malfunction detections before triggering an alert.

{'x': 1156, 'y': 741, 'width':
220, 'height': 76,
← 'confidence':
0.6368409395217896,
'class': 'blister', 'class_id': 0}

```
def send_email_alert():  
    msg = MIMEText('Malfunction detected on the product line!')  
    msg['Subject'] = 'Product Line Alert'  
    msg['From'] = 'your_email@example.com'  
    msg['To'] = 'supervisor@example.com'  
  
    with smtplib.SMTP('smtp.example.com', 587) as server:  
        server.login('your_email@example.com', 'password')  
        server.sendmail('your_email@example.com', 'supervisor@example.com', msg.as_string())
```

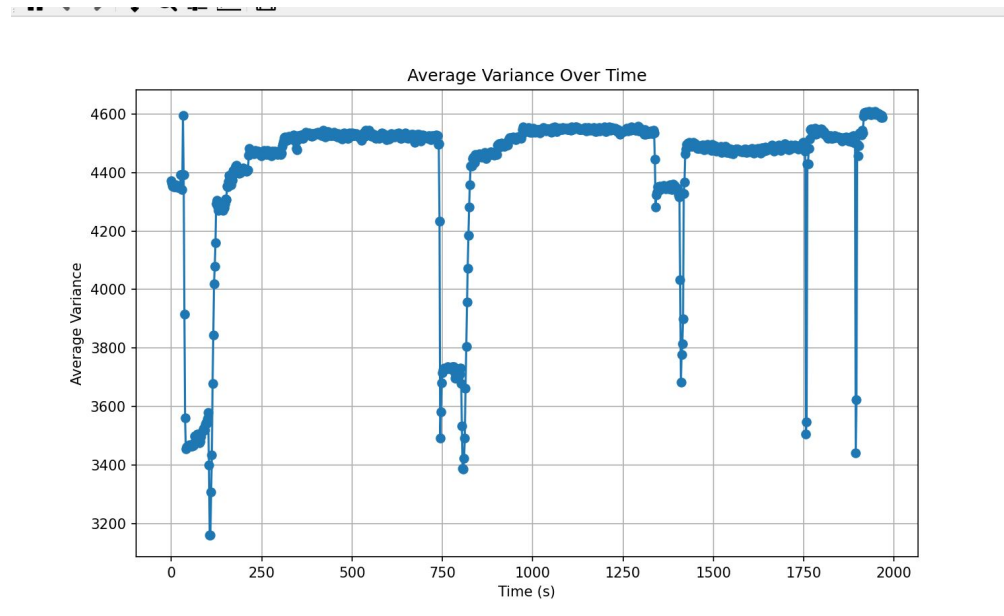
Video - Training Customized Model for Blisters Detection

- Model Training
 - Split the video into frames and label the blisters as dataset
 - Split the dataset into training set(70%), validation set(20%), testing set(10%)
 - Train a customized Yolo model in Roboflow
- Model Performance
 - Precision($TP/TP+FP$) = 65.5%
 - Recall($TP/TP+FN$) = 30.7%
 - Average Precision Metric = 59.3%

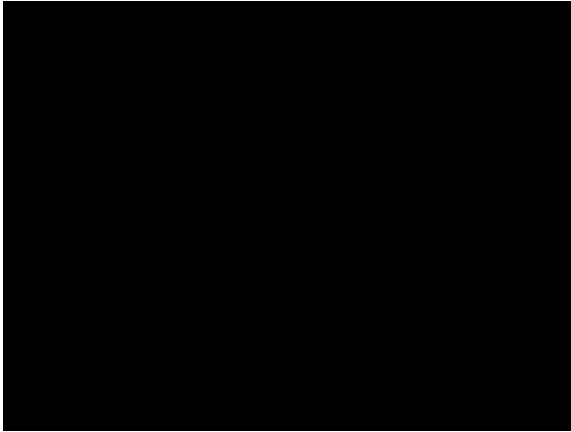


Blockers and Possible Solutions

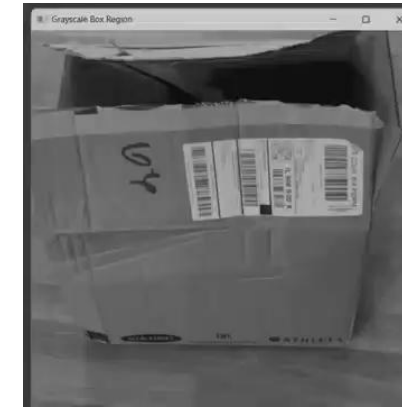
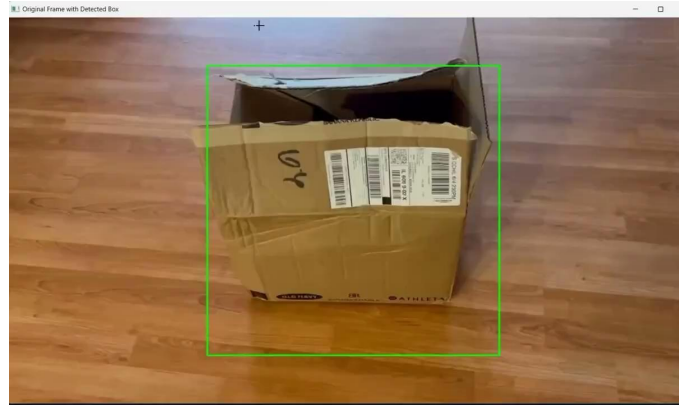
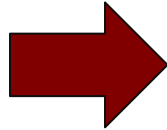
- Model Accuracy is low
 - We will try to tune the hyperparameter or fit another types of model, and increase the size of dataset.
 - We may need high quality video or we can find algorithm to improve image and video quality.
- Instead of the blisters video, we are unable to observe where and when the malfunctions and damages are on the product line. We need video with annotation of when the damages are. For example the wave happened at 30:00 - 30:05.
- Since the video is very long, when we try to calculate video variance in Azure, the calculation exceeds the pricing tier.
 - We need short and accurate video of damages.
 - We may need to increase the computational power of our Azure.
- We try to skip frame and calculate the times series of the variance, but it fails to detect abnormality on the product line.



Variance Estimation from Box Videos using OpenCV



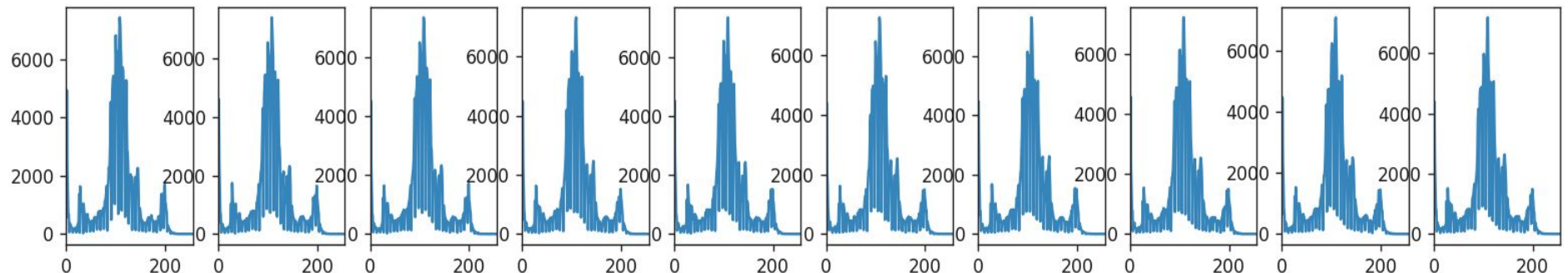
Original Video of the box



Used OpenCV to process the video frame-by-frame, focusing on a specific region (the box). It extracts and analyzes the intensity of pixels in this region to identify any anomalies or defects.

This histograms show the distribution of pixel intensities within the box region.

More peaks in the histogram correspond to more variance and hence, more chances of defects in that part of the box.



Variance Matrix

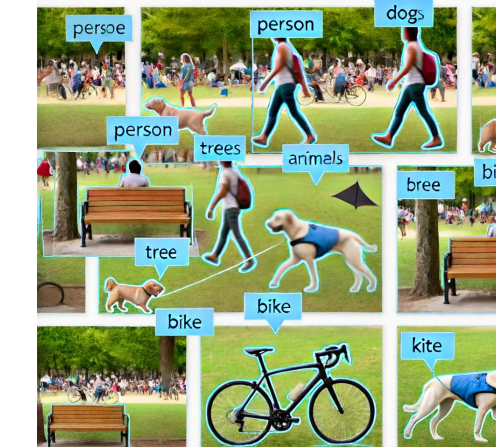
Variance of Features: [9.66820996e+03 2.04732910e+04 5.19163965e+03 4.45281006e+03

```
6.84141016e+03 5.23195996e+03 6.52839966e+02 2.12995996e+03
1.28903992e+03 9.44759949e+02 1.85449997e+02 3.31639984e+02
3.11010010e+02 4.26100044e+01 1.26159996e+02 3.08610016e+02
6.18900032e+01 6.71450012e+02 6.54159973e+02 3.00809998e+02
2.42400017e+01 3.53000000e+02 5.88489990e+02 4.60360016e+02
1.82609985e+02 7.19010010e+02 1.06582090e+04 1.71210007e+02
4.71360010e+03 3.05521021e+03 1.56380005e+03 8.88409973e+02
2.92695996e+03 1.38840002e+03 4.52489990e+02 1.65421021e+03
4.50695996e+03 7.69609985e+02 9.28489868e+02 9.04559937e+02
1.04460999e+03 8.29599991e+01 2.50839996e+02 4.39839996e+02
1.28709009e+03 7.72810059e+02 3.83160004e+02 2.25640015e+02
3.59599991e+01 4.37440002e+02 2.80690002e+02 3.25289978e+02
7.31559937e+02 3.63250000e+02 6.15359985e+02 3.87600021e+01
1.05780994e+03 6.21640015e+02 4.33640045e+02 4.93399994e+02
1.01756006e+03 3.16450012e+02 4.59960022e+02 7.75239990e+02
9.51559937e+02 1.14941003e+03 1.87776001e+03 4.31410004e+02
8.08209961e+02 2.45439987e+02 1.66508984e+03 4.43439941e+02
1.58624011e+03 1.26761011e+03 6.03650024e+02 1.12468994e+03
3.42640015e+02 7.08640015e+02 1.98569006e+03 3.70201025e+03
6.49599976e+02 4.41545020e+03 5.10061035e+03 1.76639999e+02
1.61995996e+03 2.15339990e+03 1.22607598e+04 2.18184009e+03
6.48004004e+03 3.67732930e+04 3.45330117e+04 1.08682402e+04
1.94569590e+04 7.41443994e+03 2.64228086e+04 7.82412969e+04
1.63612393e+04 3.15388125e+04 5.20429004e+03 1.82756094e+04
1.78302203e+05 1.13416387e+04 6.45628438e+04 9.11389062e+03
3.03822402e+04 2.09081006e+03 1.17880898e+04 4.45998086e+04
2.47740391e+04 8.34724023e+03 4.72052422e+04 7.25720391e+04
6.45520020e+03 1.57480908e+04 5.67104883e+04 6.61748906e+04
1.53838389e+04 1.79604902e+04 4.65383984e+03 3.54260010e+03
8.72249023e+03 8.42240039e+03 9.72056055e+03 6.00481006e+03
7.39704980e+03 1.41868982e+03 2.06781006e+03 9.93245020e+03
4.37335986e+03 1.88265996e+04 9.38020020e+03 7.07575625e+04
1.70121602e+04 3.85759979e+02 7.45083984e+03 1.92804102e+04
3.76962383e+04 2.34481992e+04 2.54932402e+04 3.79428979e+03
2.20160004e+02 4.03855547e+04 5.56646367e+04 2.68062871e+04
1.07722910e+04 4.79000000e+03 3.71469019e+03 1.18809998e+02
7.87348975e+03 3.79360010e+03 3.48539990e+03 6.53688965e+03
2.91836011e+03 2.12564990e+03 2.55239990e+02 6.48160034e+02
2.96463989e+03 2.28983984e+03 2.18820996e+03 1.54019995e+03
```

```
1.82023987e+03 1.32160004e+02 1.31003992e+03 1.35305005e+03
1.91759998e+03 6.84010071e+02 4.43399994e+02 5.21090027e+02
3.58809998e+02 4.68010010e+02 1.07360999e+03 6.20250000e+02
1.43296008e+03 2.85160004e+02 9.94650024e+02 7.23010071e+02
1.57649994e+02 3.41840027e+02 2.76840027e+02 2.10009979e+02
4.69690002e+02 6.15839966e+02 6.48760010e+02 1.96090012e+02
2.10760010e+02 6.12039978e+02 9.28599976e+02 1.47816016e+03
5.42316016e+03 6.61068994e+03 4.29049988e+02 5.50323975e+03
2.24702891e+04 7.98381104e+03 6.39860010e+03 3.64023975e+03
9.37995996e+03 4.58776025e+03 1.28641003e+03 7.82879980e+03
2.42144995e+03 4.21960022e+02 1.54744995e+03 1.94700000e+03
6.78809998e+02 2.01381018e+03 9.51640015e+02 5.67160034e+02
1.39160004e+02 8.67599945e+01 1.16889999e+02 1.20399990e+01
7.75600052e+01 3.36399994e+01 1.05010010e+02 5.06100006e+01
4.41599998e+01 2.12500000e+01 7.49000025e+00 1.32900009e+01
1.56100006e+01 2.36099987e+01 3.50900040e+01 4.64000034e+00
9.28999901e+00 0.00000000e+00 3.61000013e+00 5.03999996e+00
2.00999999e+00 7.59999990e-01 2.24000025e+00 6.09999955e-01
0.00000000e+00 2.89999992e-01 2.40000010e-01 0.00000000e+00
8.99999887e-02 1.59999996e-01 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 8.99999887e-02
8.99999887e-02 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 8.99999887e-02]
```

Microsoft Azure Vision Option

- Image Analysis 4.0: The Image Analysis service can extract features like adults and objects, allows custom model training for image classification or object detection, and generates human-readable language captions.
- Video Analysis: It can detect the presence and movements of people, count individuals entering a space, and calculate distances between them, with customizable models for target objects. It also allows the creation of a searchable index to find videos using natural language queries.
- Image tagging: Azure Computer Vision can tag thousands of recognizable objects, beings, scenery, and actions in images. This includes not just the main subject but also the setting and various items like furniture, tools, plants, animals, and gadgets



Accomplishments & Next Steps



Early Work on Dimensions' Estimation



Early Work on Anomaly Detection from Images



Early Work on Variance Estimation from Videos



Incorporated available Cloud Options (GCP and Azure) for Computer Vision



Build Product Backlog



Take Early Work and Refine it (Early Custom Training on Corrugated Images and Artifacts)



Verify Paperwork (NDAs)



Reposition Cameras

Thank you!